MCP Setup Guide (Python)

MCP (Model Context Protocol): a standard way for AI apps (hosts) to discover and call tools, resources and prompts from servers.

Roles

- Host: the AI app (Claude Desktop, an IDE, your own agent)
- Client: the connection inside the host, one per server
- Server: exposes tools / resources / prompts (e.g. notes, GitHub, a database)
- Transports: stdio (local process) or streamable HTTP (remote)

1. Install

```
python -m venv .venv && source .venv/bin/activate
pip install "mcp==2.3.0"
```

2. Minimal server (server.py)

```
from mcp.server.mcpserver import MCPServer
mcp = MCPServer("notes")

@mcp.tool()
def add_note(text: str) -> str:
    """Add a note."""
    return f"note added: {text}"

if __name__ == "__main__":
    mcp.run()          # stdio
```

(mcp 1.x used from mcp.server.fastmcp import FastMCP; 2.x renamed it to MCPServer.)

3. Minimal client

```
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
params = StdioServerParameters(command="python", args=["server.py"])
async with stdio_client(params) as (r, w):
    async with ClientSession(r, w) as s:
        await s.initialize()
        tools = (await s.list_tools()).tools
        result = await s.call_tool("add_note", {"text": "hi"})
```

4. Connect to a desktop host

Add to the host's MCP config (e.g. Claude Desktop claude_desktop_config.json):

```
{"mcpServers": {"notes": {"command": "python", "args": ["/full/path/server.py"]}}}
```

Restart the host; the tools appear automatically.

5. Debug

- Use the MCP Inspector to call tools by hand: npx @modelcontextprotocol/inspector python server.py
- Log to stderr only - stdout is the protocol channel in stdio mode

Safety

- Least privilege: expose only what is needed; read-only by default
- Validate arguments on the server; never trust the model
- Require approval in the host for write/delete tools
- Only install servers you trust; review their code and permissions